

Tutorial 09: CPU and Memory Discipline

A practical local-server Screeps module

<docs/tutorials/09-cpu-and-memory.md>

Episode 9: The Weight of Memory

Nothing in this colony has ever cost you CPU you noticed. Nine creeps across two rooms changed that – turns out scale finds every lazy decision you made back when nobody was counting.

"Every `room.find()` call scans every object of that type in the room," your collaborator says. "Every `findClosestByPath` runs actual pathfinding. You've been calling both, every tick, for every creep, since Episode 3. It was free when you had three creeps. It is extremely not free anymore, and your bucket's about to let you know it."

Goal

Learn to see CPU as a budget instead of an afterthought, and cut the easiest waste out of the roles you've already written:

- Read `Game.cpu.getUsed()`, `Game.cpu.limit`, and `Game.cpu.bucket`
- Understand why Memory is expensive to write to and a plain object often isn't
- Build a shared `cache.js` module and use it from `role.upgrader.js`, `role.builder.js`, and `role.hauler.js`
- Confirm CPU usage actually drops

Prerequisites

Tutorial 08 is complete:

- Roles `miner`, `hauler`, `upgrader`, `builder`, `reserver`, `remoteMiner`, and `remoteHauler` are all running.
- The colony spans two rooms.

Step 1: Measure Before You Change Anything

```
Game.cpu.getUsed()
Game.cpu.limit
Game.cpu.bucket
```

`getUsed()` is how much CPU the *current* tick's execution has consumed so far – call it near the end of your loop to see the tick's real cost. `limit` is your per-tick budget. `bucket` is banked unused CPU from previous ticks; it drains when you go over `limit` and refills when you come in under it.

Add a log line to the end of `main`'s loop so you can watch this over time without querying the console every tick:

```
if (Game.time % 20 === 0) {
  console.log(`CPU: ${Game.cpu.getUsed().toFixed(2)} / ${Game.cpu.limit}, bucket: ${Game.cpu.bucket}`);
}
```

Checkpoint: a log line appears roughly every 20 ticks with real, nonzero CPU usage.

 **Screenshot placeholder:** The console log panel showing a run of CPU/bucket log lines over time – useful as a reference for what a healthy, non-climbing CPU trend actually looks like versus one that's creeping up.

Step 2: Find the Waste

Look at `role.upgrader.js` and `role.builder.js`. Both call `creep.room.find(FIND_SOURCES)` every single tick a creep needs to harvest – even though the sources in a room never move and never change. That's a recomputation of something that was already true a hundred ticks ago.

Look at `role.hauler.js`. It calls `creep.pos.findClosestByPath(FIND_STRUCTURES, { filter: ... })` every tick while hauling – scanning every structure in the room and filtering it down, every time, for every hauler.

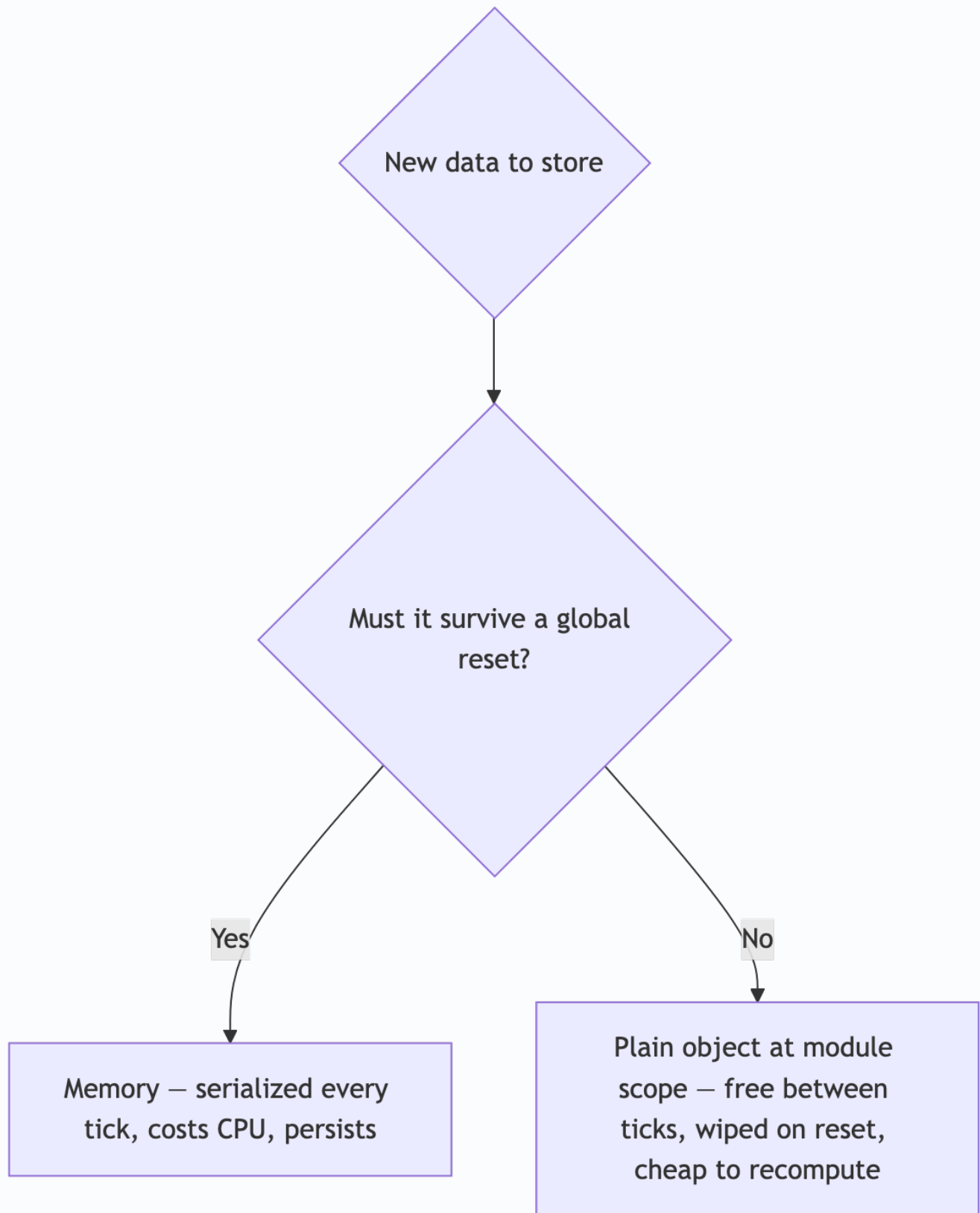
Neither of these is wrong. Both are avoidable.

Step 3: Understand Why Memory Isn't the Answer Either

The obvious fix looks like caching into `Memory` – but `Memory` is a JSON-serialized object. Every tick, the whole thing gets stringified and parsed, whether you touched it or not. Writing a large cache into `Memory` trades one CPU cost for another.

What you want instead is a plain JavaScript object declared outside the `loop` function. Sereeps keeps your script's module scope alive between ticks in the same runtime instance – a variable declared at the top of a file persists, unserialized, until a *global reset* (a code push, an uncaught exception, or a periodic engine refresh) wipes it. When that happens, the object is simply empty again, and your code recomputes it once. No crash, no special handling required – that's the property that makes this safe to rely on.

The rule going forward: if data must survive a global reset, it belongs in `Memory` (like `creep.memory.sourceId` – you need that even if the reset happens mid-life). If it's cheap to recompute and doesn't need that guarantee, keep it in a plain cache object instead.



`Memory.creeps[name].sourceId` is a **Yes**. A cached list of a room's sources is a **No** – losing it on a reset just costs one recomputation, not a broken colony.

Step 4: Build `cache.js`

Add a new file named `cache`:

```
const cache = {};

function getSources(room) {
  cache.sources = cache.sources || {};
  if (!cache.sources[room.name]) {
    cache.sources[room.name] = room.find(FIND_SOURCES).map((source) => source.id);
  }
  return cache.sources[room.name].map((id) => Game.getObjectById(id));
}

function getDeliveryTargets(room) {
  cache.deliveryTargetIds = cache.deliveryTargetIds || {};
  if (!cache.deliveryTargetIds[room.name] || Game.time % 50 === 0) {
    cache.deliveryTargetIds[room.name] = room
      .find(FIND_STRUCTURES, {
        filter: (structure) =>
          structure.structureType === STRUCTURE_EXTENSION || structure.structureType ===
STRUCTURE_SPAWN,
      })
      .map((structure) => structure.id);
  }
  return cache.deliveryTargetIds[room.name]
    .map((id) => Game.getObjectById(id))
    .filter((structure) => structure && structure.store.getFreeCapacity(RESOURCE_ENERGY) > 0);
}

module.exports = { getSources, getDeliveryTargets };
```

`getSources` never recomputes once it has an answer – source positions are permanent. `getDeliveryTargets` recomputes every 50 ticks, because which extensions exist can change (you might build more), but doesn't need to be re-scanned every single tick.

Step 5: Use the Cache in `role.upgrader.js` and `role.builder.js`

Add `const cache = require('cache');` to the top of each file, then replace:

```
const sources = creep.room.find(FIND_SOURCES);
```

with:

```
const sources = cache.getSources(creep.room);
```

The rest of both files stays the same – `sources[0]` still works, it's just no longer re-scanning the room to get it.

Step 6: Use the Cache in `role.hauler.js`

Replace the delivery-target lookup:

```
const target = creep.pos.findClosestByPath(FIND_STRUCTURES, {
  filter: (structure) =>
    (structure.structureType === STRUCTURE_EXTENSION || structure.structureType === STRUCTURE_SPAWN) &&
    structure.store.getFreeCapacity(RESOURCE_ENERGY) > 0,
});
```

with:

```
const cache = require('cache');
// ...
```

```
const candidates = cache.getDeliveryTargets(creep.room);
const target = creep.pos.findClosestByPath(candidates);
```

`findClosestByPath` still runs pathfinding – that part of the cost is inherent to picking the *nearest* target and isn't something caching removes. What you cut is the repeated `FIND_STRUCTURES` scan and filter that used to happen before the pathfinding even started.

Step 7: Confirm the Drop

Let the colony run for a few minutes with the CPU log from Step 1 active. Compare the logged values before and after this episode's changes.

Checkpoint: `Game.cpu.getUsed()` per tick should read lower on average than it did in Step 1, especially as creep count grows. The gap widens with population – this optimization matters more at nine creeps than it did at two.

Troubleshooting

If `cache.getSources` returns stale data after you thought a global reset happened, that's actually correct behavior – resets are infrequent and you can't force one from the console. To test the reset path deliberately, push any code change (even a comment) and watch the next log line; a fresh cache computation is harmless and expected.

If `require('cache')` fails, confirm the file is named exactly `cache` in the editor and that `module.exports` is an object with both functions attached, not a single default export.

If CPU usage doesn't visibly drop, check whether `Game.time % 50 === 0` is actually being hit – with a `tickRate` far from 1 real second, "every 50 ticks" may take longer to observe than you expect on this local server.

Completion Criteria

You are done when:

- `cache.js` exists and exports `getSources` and `getDeliveryTargets`.
- `role.upgrader.js`, `role.builder.js`, and `role.hauler.js` all use it instead of raw `room.find()` calls in the hot path.
- A CPU log line prints every 20 ticks.
- Logged CPU usage is measurably lower than before this episode, at comparable creep counts.

Learning Notes

After completing the tutorial, write down:

- What's the actual CPU cost difference you measured, in numbers, not just "it felt faster"?
- What's one piece of data in this codebase that *does* need to survive a global reset, and why does that make it a `Memory` candidate instead of a cache candidate?
- The `getDeliveryTargets` cache refreshes every 50 ticks on a fixed schedule. What's a smarter invalidation trigger you could use instead?
- If `Game.cpu.bucket` hit zero, what should this colony's code do differently? Does anything right now check for that?

Next: Episode 10 – Two Minds

Every optimization in this episode came from a conversation, not a solo debugging session.

"You've been working with me this whole time," your collaborator says, "but you've never had to think about *how* you were working with me. That's the next skill. Not writing better Screeps code – writing better prompts about your Screeps code. There's a version of you that still writes 'it's broken, pls help' in every bug report. Let's fix that before it's a habit."

See `docs/roadmap.md` for the full season.